

Compiladores

Análise Semântica

Ciência da Computação / 6ª etapa

Prof. Dr. Leandro Carlos Fernandes

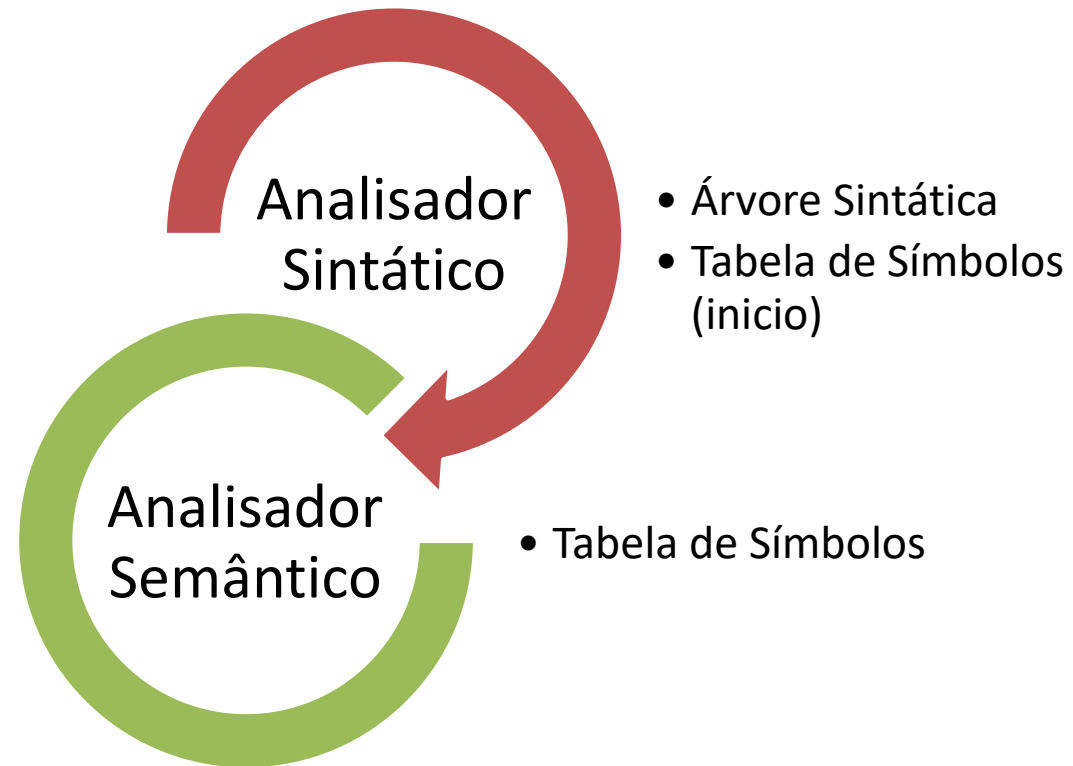


Análise Semântica

(3ª etapa da Análise)

0 1 1 0 i 0
0 0 k n o w
0 w h a t 0 0 0 0
0 0 0 y o u 1 0 1
1 0 1 m e a n . .
1 0 1 1 0 0 1 0 0

A análise semântica



Resultados da Análise Sintática ...

As técnicas de **análise sintática** vistas descrevem basicamente reconhecedores, ou seja, máquinas que determinam se uma cadeia pertence ou não à linguagem reconhecida. Entretanto:

No caso de aceitação: a informação sobre a forma de derivação de x deve ser usada na geração do código para x ; e

No caso de não aceitação: a informação disponível deve ser usada para tratamento de erros.

Sinalização, para que o programador possa corrigir o erro encontrado; e recuperação, para que o *parser* possa dar continuação à análise.



O componente semântico

Durante a compilação o analisador deve verificar a utilização adequada dos identificadores

- Análise Contextual: declaração prévia de variáveis, escopo de uso, etc.

- Checagem de tipos e compatibilidade

Note que estas tarefas estão além do domínio da sintaxe (Gram. Livres de Contexto - GLC)

- Tais questões aumentam a GLC e completam a definição do que são programas válidos.



A análise ocorre em dois momentos:

Semântica Estática

Conjunto de restrições que determinam se os programas sintaticamente corretos são válidos.

Semântica de Tempo de Execução

Usada para especificar o que o programa faz. Em outras palavras, aspectos ligados a execução.



Semântica estática

Relaciona-se ao conjunto de restrições que vão além dos aspectos sintáticos, mas que determinam o que são programas válidos.

As atividades compreendidas nesta etapa são:

- Checagem de tipos,

- Análise de escopo de declarações,

- Verificação da quantidade e dos tipos dos parâmetros em chamadas à sub-rotinas.

Pode ser especificada formalmente por uma *Gramática de Atributos*



Semântica de tempo de execução

Aspectos relacionados ao comportamento do programa, isto é, a relação que há entre o programa-fonte e a sua execução dinâmica.

Exemplos:

`L: goto L;`

Implica em loop infinito, decorrente de um salto para a própria instrução.
Alguns compiladores proíbem, outros não.

`if (i<>0) && (K/I > 10) ...`

Haverá divisão por zero caso o compilador teste todas as cláusulas, entretanto não seria necessário proceder a segunda verificação caso a primeira já fosse



Esta etapa considera questões importantes para a fase de geração de código

Permite, por exemplo, uma organização diferenciada das instruções de máquina.

Muitas vezes é o compilador quem serve de definição para a linguagem quando esta não está totalmente especificada.

Geralmente esta componente é especificada de maneira informal, mas também pode ser feito através do uso de formalismos (tais com as Gramáticas de Atributos).



Tabela de Símbolos



Armazenando informações sobre os identificadores ...

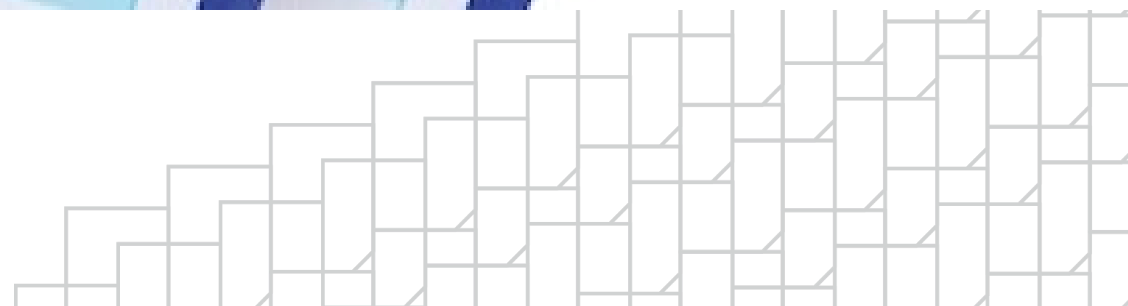


Tabela de Símbolos

Armazena as informações sobre todos os identificadores do código fonte

- Captura a sensibilidade ao contexto e as ações executadas no decorrer do programa

Está atrelada a todas as etapas da compilação, sendo a estrutura principal do processo.

E por sua vez, é fundamental para:

- Realizar a análise semântica

- A geração de código



Operações com a Tabela de Símbolos

Podem ser implementadas como:

1. Diretamente na **Análise Sintática**

Inserção: na análise declarações de variáveis, sub-rotinas, parâmetros ...

Busca: em atribuições, expressões, chamadas de sub-rotinas ou qualquer outro uso de um identificador em um bloco de comandos.

2. Chamadas na **Gramática de Atributos**

```
L0 → id, L1      if (buscaTS(id) == false)
                     incluirTS(id, L0.tipo)
                     else
                       ERRO("Identificador já declarado")
```



Tarefas da Análise Semântica

Responsável fundamentalmente por três tarefas:

1. Construir a descrição interna dos **tipos** e **estruturas de dados** definidos no programa do usuário;
2. Armazenar na **Tabela Símbolos** as **informações sobre os identificadores** (de constante, tipos, variáveis, procedimentos, parâmetros e funções) que são usados no programa;
3. Verificar o programa quanto a **erros semânticos** (erros dependentes de contexto) e checagens de tipos com base nas informações contidas na Tabela de Símbolos.



1. Representação de tipos e estruturas de dados definidos pelo programador

As linguagens modernas oferecem um grande repertório de tipos e também permitem que o programador especifique seus próprios tipos de dados.

Ao compilador cabe:

- representar as especificações dos tipos

- e usar tais informações para a previsão do uso de memória em tempo de execução, pelos objetos que forem declarados como sendo de um tipo especificado.



2. Armazenar informações sobre os identificadores na Tabela de Símbolos

Uma Tabela de Símbolos reflete a estrutura do programa, pois guarda informações sobre o escopo de identificadores.

À medida que o compilador processa o programa fonte encontra os identificadores definidos e também os utilizados pelo programa.

Por exemplo, em situações como declarações de variáveis ou de procedimentos com seus parâmetros.

Para cada referência, o compilador terá necessidade de conhecer os atributos correspondentes (que sejam de interesse para a geração de código e verificação de erros semânticos).

Por exemplo, no caso de variáveis, poderiam ser a categoria, tipo e endereço na área de dados.



3. Verificar quanto a erros semânticos dependentes de contexto e de tipos

Identificador já declarado no escopo (nível) atual

Tipo não definido

Limite inferior > limite superior na declaração de vetores/matrizes

Função não declarada

Função, variável, parâmetro, ou constante não definidas (checagem no lado direito de atribuições)

Incompatibilidade no número de parâmetros

Procedimento não declarado

Função, variável, procedimento ou parâmetros não definidos (lado esquerdo de atribuições/lado esquerdo)

Identificador de tipo esperado





Universidade Presbiteriana
Mackenzie



Faculdade de
Computação e Informática

